Manuel DevOps

Table des Matières

1. **Ligne de Commande Linux**
    - Commandes Linux de Base
    - Commandes Linux Intermédiaires
    - Commandes Linux Avancées
    - Commandes Réseau
    - Gestion et Recherche de Fichiers
    - Surveillance Système
    - Gestion des Paquets
    - Disque et Système de Fichiers
    - Scripts et Automatisation
    - Développement et Débogage
    - Autres Commandes Utiles
    - Scénarios et Scripts du Manuel Linux
2. **Contrôle de Version Git**
    - Commandes Git de Base
    - Branches et Fusion
    - Dépôts Distants
    - Stash et Nettoyage
    - Étiquetage
    - Commandes Avancées
    - Commandes Spécifiques GitHub
    - Workflows du Manuel Git
3. **Conteneurisation Docker**
    - Commandes Docker de Base
    - Commandes Docker Intermédiaires
    - Commandes Docker Avancées
    - Exemples du Manuel Docker
4. **Orchestration Kubernetes (K8s)**
    - Commandes Kubernetes de Base
    - Commandes Kubernetes Intermédiaires
    - Commandes Kubernetes Avancées
    - Exemples du Manuel Kubernetes (Manifestes YAML)

5. **Helm (Le Gestionnaire de Paquets K8s)**
    - Commandes Helm de Base
    - Installation et Mise à Niveau des Charts
    - Travail avec les Charts Helm
    - Commandes Helm Avancées
    - Dépôts de Charts Helm
    - Valeurs et Personnalisation Helm
    - Template et Débogage Helm
    - Intégration Helm et Kubernetes
    - Développement de Charts Helm
    - Helm avec CLI Kubernetes
    - Dépendances des Charts Helm
    - Historique et Retours Arrière Helm
    - Workflow du Manuel Helm
6. **Terraform (Infrastructure as Code)**
    - Commandes Terraform de Base
    - Exemple du Manuel Terraform (HCL)
7. **Ansible (Gestion de Configuration)**
    - Concepts Fondamentaux
    - Commandes Courantes
    - Exemple de Playbook Ansible (YAML)
8. **CI/CD (Intégration/Déploiement Continu)**
    - Concepts Fondamentaux (GitHub Actions)
    - Exemple du Manuel CI/CD (GitHub Actions)
9. **Surveillance et Observabilité**
    - Outils Principaux
    - Prometheus & PromQL
    - Grafana
    - Journalisation (ELK/EFK)

---

1. Ligne de Commande Linux

Linux est le fondement des opérations DevOps. Ces commandes vous aident à naviguer dans les systèmes, gérer les fichiers, configurer les permissions et automatiser les tâches.

Commandes Linux de Base

1. **pwd** - Afficher le répertoire de travail actuel
2. **ls** - Lister les fichiers et répertoires
   - `ls -la` - Lister tous les fichiers (y compris cachés) en format long
3. **cd** - Changer de répertoire
   - `cd ..` - Remonter d'un répertoire
   - `cd ~` ou `cd` - Aller au répertoire personnel
4. **touch** - Créer un fichier vide
5. **mkdir** - Créer un nouveau répertoire
   - `mkdir -p /chemin/vers/repertoire/imbriqué` - Créer les répertoires parents si nécessaire
6. **rm** - Supprimer des fichiers ou répertoires
   - `rm -f <fichier>` - Forcer la suppression d'un fichier
   - `rm -r <rep>` - Supprimer récursivement un répertoire
   - `rm -rf <rep>` - Forcer et supprimer récursivement un répertoire
7. **rmdir** - Supprimer les répertoires vides
8. **cp** - Copier des fichiers ou répertoires
   - `cp <source> <destination>`
   - `cp -r <rep_source> <rep_destination>` - Copier récursivement un répertoire
9. **mv** - Déplacer ou renommer des fichiers et répertoires
   - `mv <ancien_nom> <nouveau_nom>` - Renommer
   - `mv <source> <rep_destination>` - Déplacer
10. **cat** - Afficher le contenu d'un fichier
11. **echo** - Afficher une ligne de texte
    - `echo "texte" > fichier.txt` - Écraser le fichier avec le texte
    - `echo "texte" >> fichier.txt` - Ajouter le texte au fichier
12. **clear** - Effacer l'écran du terminal

Commandes Linux Intermédiaires

13. **chmod** - Modifier les permissions de fichier
    - `chmod +x script.sh` - Rendre un script exécutable
    - `chmod 755 <fichier>` - Définir les permissions (Propriétaire: rwx, Groupe: r-x, Autres: r-x)
14. **chown** - Changer la propriété d'un fichier
    - `chown <utilisateur>:<groupe> <fichier>`
15. **find** - Rechercher des fichiers et répertoires
    - `find . -name "*.txt"` - Trouver tous les fichiers se terminant par .txt
16. **grep** - Rechercher du texte dans un fichier
    - `grep "erreur" /var/log/syslog` - Rechercher "erreur" dans un fichier
    - `grep -r "texte" /chemin/vers/rep` - Rechercher récursivement "texte"

17. **wc** - Compter les lignes, mots et caractères
    - `wc -l <fichier>` - Compter uniquement les lignes
18. **head** - Afficher les premières lignes d'un fichier
    - `head -n 20 <fichier>` - Afficher les 20 premières lignes
19. **tail** - Afficher les dernières lignes d'un fichier
    - `tail -n 20 <fichier>` - Afficher les 20 dernières lignes
    - `tail -f <fichier>` - Suivre le fichier et afficher les nouvelles lignes en temps réel
20. **sort** - Trier le contenu d'un fichier
21. **uniq** - Supprimer les lignes en double (nécessite une entrée triée)
    - `sort fichier.txt | uniq -c` - Compter les lignes uniques
22. **diff** - Comparer deux fichiers ligne par ligne
23. **tar** - Archiver des fichiers dans un tarball
    - `tar -cvf archive.tar /chemin/vers/rep` - Créer
    - `tar -xvf archive.tar` - Extraire
    - `tar -czvf archive.tar.gz /chemin/vers/rep` - Créer et compresser avec gzip
    - `tar -xzvf archive.tar.gz` - Extraire une archive gzippée
24. **zip/unzip** - Compresser et extraire des fichiers ZIP
25. **df** - Afficher l'utilisation de l'espace disque
    - `df -h` - Format lisible par l'humain
26. **du** - Afficher la taille d'un répertoire
    - `du -sh /chemin/vers/rep` - Résumé lisible de la taille totale
27. **top** - Surveiller les processus système en temps réel
28. **ps** - Afficher les processus actifs
    - `ps aux` - Afficher tous les processus de tous les utilisateurs
29. **kill** - Terminer un processus par son PID
    - `kill <PID>` - Arrêt gracieux (SIGTERM)
    - `kill -9 <PID>` - Forcer l'arrêt (SIGKILL)
30. **ping** - Vérifier la connectivité réseau
31. **wget** - Télécharger des fichiers depuis Internet
32. **curl** - Transférer des données depuis ou vers un serveur
    - `curl -L <url>` - Suivre les redirections
    - `curl -o <fichier> <url>` - Télécharger dans un fichier
33. **scp** - Copier des fichiers de manière sécurisée entre systèmes
    - `scp <fichier_local> <utilisateur>@<hôte_distant>:/chemin/`
34. **rsync** - Synchroniser des fichiers et répertoires
    - `rsync -avz <rep_source>/ <utilisateur>@<hôte_distant>:/chemin/`

Commandes Linux Avancées

35. **awk** - Traitement de texte et analyse de motifs
36. **sed** - Éditeur de flux pour filtrer et transformer du texte
    - `sed 's/foo/bar/g' fichier.txt` - Remplacer "foo" par "bar" globalement
37. **cut** - Supprimer des sections de chaque ligne d'un fichier
38. **tr** - Traduire ou supprimer des caractères
39. **xargs** - Construire et exécuter des lignes de commande depuis l'entrée standard
40. **ln** - Créer des liens symboliques ou physiques
    - `ln -s /chemin/vers/original /chemin/vers/lien` - Créer un lien symbolique
41. **free** - Afficher l'utilisation de la mémoire
    - `free -h` - Format lisible
42. **iostat** - Afficher les statistiques CPU et I/O
43. **netstat** - Statistiques réseau
    - `netstat -tulnp` - Afficher les ports TCP/UDP en écoute et les programmes
44. **ifconfig/ip** - Configurer les interfaces réseau
    - `ip addr show` - (Moderne) Afficher les adresses IP
45. **iptables** - Configurer les règles de pare-feu
46. **systemctl** - Contrôler le gestionnaire système systemd
    - `systemctl start <service>`
    - `systemctl stop <service>`
    - `systemctl restart <service>`
    - `systemctl status <service>`
    - `systemctl enable <service>` - Démarrer au boot
    - `systemctl disable <service>` - Ne pas démarrer au boot
47. **journalctl** - Afficher les journaux système (de systemd)
    - `journalctl -u <service>` - Afficher les journaux d'un service spécifique
    - `journalctl -f` - Suivre tous les journaux
48. **crontab** - Planifier des tâches récurrentes
    - `crontab -e` - Éditer le crontab de l'utilisateur
49. **at** - Planifier des tâches pour une heure spécifique
50. **uptime** - Afficher le temps de fonctionnement du système
51. **whoami** - Afficher l'utilisateur actuel
52. **users** - Lister tous les utilisateurs actuellement connectés
53. **hostname** - Afficher ou définir le nom d'hôte du système
54. **env** - Afficher les variables d'environnement
55. **export** - Définir des variables d'environnement

Commandes Réseau

56. **ip addr** - Afficher ou configurer les adresses IP
57. **ip route** - Afficher ou manipuler les tables de routage
58. **traceroute** - Tracer la route des paquets vers un hôte

59. **nslookup** - Interroger les enregistrements DNS
60. **dig** - Interroger les serveurs DNS (plus détaillé que nslookup)
61. **ssh** - Se connecter à un serveur distant via SSH
    - `ssh <utilisateur>@<hôte>`
    - `ssh -i /chemin/vers/cle.pem <utilisateur>@<hôte>` - Utiliser un fichier d'identité
62. **ftp** - Transférer des fichiers via le protocole FTP
63. **nmap** - Analyse et découverte réseau
64. **telnet** - Communiquer avec des hôtes distants
65. **netcat (nc)** - Lire/écrire des données sur les réseaux

Gestion et Recherche de Fichiers

66. **locate** - Trouver des fichiers rapidement via une base de données
67. **stat** - Afficher des informations détaillées sur un fichier
68. **tree** - Afficher les répertoires sous forme d'arbre
69. **file** - Déterminer le type d'un fichier
70. **basename** - Extraire le nom de fichier d'un chemin
71. **dirname** - Extraire la partie répertoire d'un chemin

Surveillance Système

72. **vmstat** - Afficher les statistiques de mémoire virtuelle
73. **htop** - Visualiseur de processus interactif (alternative à top)
74. **lsof** - Lister les fichiers ouverts
    - `lsof -i :<port>` - Trouver quel processus utilise un port spécifique
75. **dmesg** - Afficher les messages du tampon en anneau du noyau
76. **iotop** - Afficher les I/O disque en temps réel par processus

Gestion des Paquets

77. **apt (Debian/Ubuntu)** - Gestionnaire de paquets
    - `apt update`
    - `apt upgrade`
    - `apt install <paquet>`
    - `apt remove <paquet>`
78. **yum/dnf (RHEL/CentOS/Fedora)** - Gestionnaire de paquets
    - `dnf install <paquet>`
79. **snap** - Gérer les paquets snap
80. **rpm** - Gérer les paquets RPM

Disque et Système de Fichiers

81. **mount/umount** - Monter ou démonter des systèmes de fichiers
82. **fsck** - Vérifier et réparer les systèmes de fichiers
83. **mkfs** - Créer un nouveau système de fichiers
84. **blkid** - Afficher des informations sur les périphériques de bloc
85. **lsblk** - Lister les informations sur les périphériques de bloc
86. **parted** - Gérer les partitions de manière interactive

Scripts et Automatisation

87. **bash** - Interpréteur de commandes et shell de script
88. **sh** - Interpréteur shell historique
89. **alias** - Créer des raccourcis pour les commandes
90. **source** - Exécuter des commandes depuis un fichier dans le shell actuel

Développement et Débogage

91. **gcc** - Compiler des programmes C
92. **make** - Construire et gérer des projets
93. **strace** - Tracer les appels système et les signaux
94. **gdb** - Déboguer des programmes
95. **git** - Système de contrôle de version
96. **vim/nano** - Éditeurs de texte pour scripts et édition

Autres Commandes Utiles

97. **date** - Afficher ou définir la date et l'heure du système
98. **cal** - Afficher un calendrier
99. **man** - Afficher le manuel d'une commande
100. **history** - Afficher les commandes précédemment exécutées

Scénarios et Scripts du Manuel Linux

Scénarios Courants

Trouver quel processus utilise le port 80:

```
sudo lsof -i :80
# OU
sudo netstat -tulnp | grep :80
```

Trouver les 10 fichiers/répertoires les plus volumineux:

```
# Trouver les 10 plus grands répertoires dans /
sudo du -Sh / | sort -rh | head -n 10
```

```
# Trouver les 10 plus grands fichiers dans /
sudo find / -type f -exec du -Sh {} + | sort -rh | head -n 10
```

Surveiller en direct un fichier journal pour une erreur spécifique:

```
tail -f /var/log/app.log | grep -i 'ERROR'
```

Trouver tous les fichiers modifiés dans les 2 derniers jours:

```
find /chemin/vers/recherche -mtime -2
```

Compter les IP uniques accédant à un serveur web (depuis le journal d'accès):

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -n 20
```

Exemple de Script Bash (backup.sh)

Un script simple pour sauvegarder un répertoire.

```
#!/bin/bash
# Définir les variables
SRC_DIR="/var/www/html"
DEST_DIR="/mnt/backups/web"
DATE=$(date +%Y-%m-%d-%H%M%S)
BACKUP_FILE="$DEST_DIR/web_backup_$DATE.tar.gz"

# Créer le répertoire de sauvegarde s'il n'existe pas
mkdir -p $DEST_DIR

# Créer l'archive compressée
echo "Démarrage de la sauvegarde de $SRC_DIR..."
tar -czvf $BACKUP_FILE $SRC_DIR

# Vérifier si la sauvegarde a réussi
if [ $? -eq 0 ]; then
   echo "Sauvegarde réussie! Fichier: $BACKUP_FILE"
else
   echo "ÉCHEC de la sauvegarde."
fi

# Optionnel: Supprimer les sauvegardes de plus de 7 jours
```

```
find $DEST_DIR -name "web_backup_*.tar.gz" -mtime +7 -exec rm {} \;
echo "Anciennes sauvegardes nettoyées."
```

Pour l'exécuter:

```
chmod +x backup.sh
./backup.sh
```

---

2. Contrôle de Version Git

Git est votre machine à remonter le temps pour le code. Il suit chaque modification, permet la collaboration en équipe et vous permet d'annuler les erreurs.

Commandes Git de Base

1. **git init** - Initialise un nouveau dépôt Git
2. **git clone <url>** - Copie un dépôt distant
3. **git status** - Affiche l'état du répertoire de travail
4. **git add <fichier>** - Ajoute des modifications à la zone de staging
   - `git add .` - Mettre en staging toutes les modifications
5. **git commit -m "Message"** - Enregistre les modifications dans le dépôt
6. **git config** - Configure les paramètres utilisateur
   - `git config --global user.name "Votre Nom"`
   - `git config --global user.email "vous@exemple.com"`

Journal et Différences Git

7. **git log** - Affiche l'historique des commits
   - `git log --oneline --graph --decorate` - Vue plus propre et graphique
8. **git show <hash-commit>** - Affiche les détails d'un commit spécifique
9. **git diff** - Montre les changements entre commits, staging et répertoire de travail
   - `git diff` - Changements dans le répertoire de travail vs. zone de staging
   - `git diff --staged` - Changements dans la zone de staging vs. dernier commit
10. **git reset** - Retire des modifications du staging ou réinitialise des commits
    - `git reset HEAD <fichier>` - Retirer un fichier du staging
    - `git reset --hard <hash-commit>` - DANGEREUX: Annuler tous les changements jusqu'à un commit spécifique

Branches et Fusion

11. **git branch** - Liste les branches ou crée une nouvelle branche
    - `git branch <nom-branche>` - Créer une nouvelle branche
    - `git branch -d <nom-branche>` - Supprimer une branche
12. **git checkout <nom-branche>** - Bascule vers une branche
    - `git checkout -b <nom-branche>` - Créer une nouvelle branche et basculer dessus
13. **git switch <nom-branche>** - (Moderne) Bascule entre branches
14. **git merge <nom-branche>** - Combine les changements d'une branche dans une autre
15. **git rebase <nom-branche>** - Déplace ou combine des commits d'une branche sur une autre
16. **git cherry-pick <hash-commit>** - Applique des commits spécifiques d'une branche à une autre

Dépôts Distants

17. **git remote** - Gère les connexions aux dépôts distants
    - `git remote add <nom> <url>` (ex: `git remote add origin ...`)
18. **git push <distant> <branche>** - Envoie les modifications vers un dépôt distant
    - `git push -u origin <nom-branche>` - Pousser et définir l'upstream
19. **git pull <distant> <branche>** - Récupère et fusionne les changements d'un distant
20. **git fetch <distant>** - Télécharge les changements d'un distant sans fusionner
21. **git remote -v** - Liste les URLs des dépôts distants

Stash et Nettoyage

22. **git stash** - Sauvegarde temporairement les modifications non commitées
23. **git stash pop** - Applique les modifications stashées et les supprime de la liste
24. **git stash list** - Liste tous les stashes
25. **git clean -f** - Supprime les fichiers non suivis du répertoire de travail
    - `git clean -fd` - Supprime aussi les répertoires non suivis

Étiquetage

26. **git tag -a <nom-tag> -m "Message"** - Crée un tag annoté
27. **git tag -d <nom-tag>** - Supprime un tag
28. **git push --tags** - Pousse les tags vers un dépôt distant

Commandes Git Avancées

29. **git bisect** - Trouve le commit qui a introduit un bug
30. **git blame <fichier>** - Montre quel commit et auteur a modifié chaque ligne
31. **git reflog** - Affiche un journal des changements de HEAD (utile pour récupérer des commits perdus)

32. **git submodule** - Gère les dépôts externes en tant que sous-modules
33. **git archive** - Crée une archive des fichiers du dépôt
34. **git gc** - Nettoie les fichiers inutiles et optimise le dépôt

Commandes Spécifiques GitHub (utilisant gh CLI)

35. **gh auth login** - Se connecte à GitHub via la ligne de commande
36. **gh repo clone <utilisateur/depot>** - Clone un dépôt GitHub
37. **gh issue list** - Liste les issues dans un dépôt GitHub
38. **gh pr create** - Crée une pull request sur GitHub
39. **gh repo create** - Crée un nouveau dépôt GitHub

Workflows du Manuel Git

Workflow 1: Démarrer une Nouvelle Fonctionnalité (GitHub Flow)

1. Synchroniser votre branche main:

```
git switch main
git pull origin main
```

2. Créer votre branche de fonctionnalité:

```
git switch -b feature/ma-nouvelle-chose
```

3. Faire votre travail (éditer des fichiers, etc.)

4. Ajouter et committer vos modifications:

```
git add .
git commit -m "Feat: Ajouter un composant pour ma-nouvelle-chose"
```

5. Pousser votre branche vers le distant:

```
git push -u origin feature/ma-nouvelle-chose
```

6. Créer une Pull Request: Aller sur GitHub pour ouvrir une PR de votre branche vers main

Workflow 2: Annuler une Erreur

Cas A: Vous venez de committer, mais voulez changer le message

git commit --amend -m "Un meilleur message de commit"

Cas B: Vous voulez ajouter plus de fichiers au dernier commit

git add nouveau-fichier.txt
git commit --amend --no-edit

Cas C: Vous voulez complètement annuler le dernier commit (et garder les changements)

git reset --soft HEAD~1
Vos fichiers sont inchangés, le commit est annulé

Cas D: Vous voulez annuler un commit déjà public (poussé)

N'utilisez PAS git reset! Utilisez git revert
git revert <hash-commit-à-annuler>
Cela crée un *nouveau* commit qui est l'inverse du mauvais
git push

---

3. Conteneurisation Docker

Docker emballe les applications dans des conteneurs portables. Ces commandes aident à construire, expédier et exécuter des applications de manière cohérente.

Commandes Docker de Base

1. **docker --version** - Affiche la version Docker installée
2. **docker info** - Affiche les informations système sur Docker
3. **docker pull image:tag** - Télécharge une image
4. **docker images** - Liste toutes les images téléchargées
5. **docker run <image>** - Crée et démarre un nouveau conteneur
   - `docker run -it <image> bash` - Exécuter interactivement avec un shell
   - `docker run -d -p 8080:80 <image>` - Exécuter détaché, mappant le port 8080 (hôte) vers 80 (conteneur)
6. **docker ps** - Liste les conteneurs en cours d'exécution
7. **docker ps -a** - Liste tous les conteneurs (en cours et arrêtés)
8. **docker stop <id_conteneur_ou_nom>** - Arrête un conteneur en cours
9. **docker start <id_conteneur_ou_nom>** - Démarre un conteneur arrêté
10. **docker rm <id_conteneur_ou_nom>** - Supprime un conteneur

    - `docker rm $(docker ps -aq)` - Supprimer tous les conteneurs arrêtés
11. **docker rmi <id_image_ou_nom>** - Supprime une image
12. **docker exec -it <conteneur> <commande>** - Exécute une commande dans un conteneur en cours

Commandes Docker Intermédiaires

13. **docker build -t <nom_image:tag> .** - Construit une image depuis un Dockerfile
14. **docker commit <conteneur> <nouvelle_image:tag>** - Crée une image à partir des changements d'un conteneur
15. **docker logs <conteneur>** - Récupère les journaux d'un conteneur
    - `docker logs -f <conteneur>` - Suivre les journaux
16. **docker inspect <conteneur_ou_image>** - Retourne des informations détaillées
17. **docker stats** - Affiche les statistiques d'utilisation des ressources en direct
18. **docker cp <chemin_src> <conteneur>:<chemin_dest>** - Copie des fichiers
    - `docker cp <conteneur>:<chemin_src> <chemin_dest>`
19. **docker rename <ancien_nom> <nouveau_nom>** - Renomme un conteneur
20. **docker network ls** - Liste tous les réseaux Docker
21. **docker network create <nom_réseau>** - Crée un nouveau réseau
22. **docker network inspect <nom_réseau>** - Affiche les détails d'un réseau
23. **docker network connect <réseau> <conteneur>** - Connecte un conteneur à un réseau
24. **docker volume ls** - Liste tous les volumes Docker
25. **docker volume create <nom_volume>** - Crée un nouveau volume
26. **docker volume inspect <nom_volume>** - Fournit des détails sur un volume
27. **docker volume rm <nom_volume>** - Supprime un volume

Commandes Docker Avancées

28. **docker-compose up** - Démarre les services définis dans docker-compose.yml
    - `docker-compose up -d` - Exécuter en mode détaché
29. **docker-compose down** - Arrête et supprime les services
    - `docker-compose down -v` - Supprime aussi les volumes
30. **docker-compose logs** - Affiche les journaux des services
31. **docker-compose exec <nom_service> <commande>** - Exécute une commande dans un service
32. **docker save -o <fichier.tar> <image>** - Exporte une image vers un fichier tar
33. **docker load -i <fichier.tar>** - Importe une image depuis un fichier tar
34. **docker export <conteneur> > <conteneur.tar>** - Exporte le système de fichiers d'un conteneur
35. **docker import <conteneur.tar> <nouvelle_image>** - Crée une image depuis un conteneur exporté
36. **docker system df** - Affiche l'utilisation du disque par Docker
37. **docker system prune** - Nettoie les ressources inutilisées

    - `docker system prune -af` - Nettoyer tout (images, conteneurs, volumes) sans confirmation
38. **docker tag <ancienne_image:tag> <nouvelle_image:tag>** - Assigne un nouveau tag à une image
39. **docker push image:tag** - Téléverse une image vers un registre Docker
40. **docker login** - Se connecte à un registre Docker
41. **docker logout** - Se déconnecte d'un registre Docker
42. **docker swarm init** - Initialise un Docker Swarm
43. **docker service create** - Crée un nouveau service dans Swarm
44. **docker stack deploy -c <compose.yml> <nom_stack>** - Déploie une stack
45. **docker stack rm <nom_stack>** - Supprime une stack

Exemples du Manuel Docker

Exemple 1: Dockerfile pour une Application Node.js Simple

```
# Utiliser une image Node.js officielle comme image parent
FROM node:18-alpine

# Définir le répertoire de travail dans le conteneur
WORKDIR /usr/src/app

# Copier package.json et package-lock.json vers le répertoire de travail
COPY package*.json ./

# Installer les dépendances nécessaires
RUN npm install

# Copier le reste du code source de l'application
COPY . .

# Rendre le port 3000 disponible au monde extérieur
EXPOSE 3000

# Définir la commande pour exécuter l'application
CMD [ "node", "server.js" ]
```

Exemple 2: docker-compose.yml pour une Application Web + Base de Données

Cet exemple démarre un site WordPress et une base de données MySQL, les connectant avec un réseau et persistant les données avec des volumes.

```
version: '3.8'
```

```yaml
services:
  # Service WordPress
  wordpress:
    image: wordpress:latest
    ports:
      - "8000:80" # Mapper le port 8000 de l'hôte vers le port 80 du conteneur
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: motdepasse
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wordpress_data:/var/www/html # Persister les fichiers WordPress
    networks:
      - app_network
    depends_on:
      - db

  # Service Base de Données MySQL
  db:
    image: mysql:8.0
    restart: always
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: motdepasse
      MYSQL_ROOT_PASSWORD: motdepasseroot
    volumes:
      - db_data:/var/lib/mysql # Persister les données de la base
    networks:
      - app_network

# Définir les réseaux
networks:
  app_network:
    driver: bridge

# Définir les volumes
volumes:
  wordpress_data:
  db_data:
```

---

4. Orchestration Kubernetes (K8s)

Kubernetes automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées.

Commandes Kubernetes de Base (kubectl)

1. **kubectl version** - Affiche la version client/serveur de Kubernetes
2. **kubectl cluster-info** - Affiche les informations du cluster
3. **kubectl get nodes** - Liste tous les nœuds du cluster
4. **kubectl get pods** - Liste tous les pods dans le namespace par défaut
   - `kubectl get pods -n <namespace>` - Lister les pods dans un namespace spécifique
   - `kubectl get pods -A` - Lister les pods dans tous les namespaces
   - `kubectl get pods -o wide` - Obtenir plus de détails (IP, nœud)
5. **kubectl get services** - Liste tous les services
6. **kubectl get namespaces** - Liste tous les namespaces
7. **kubectl describe pod <nom-pod>** - Affiche des informations détaillées sur un pod
8. **kubectl logs <nom-pod>** - Affiche les journaux d'un pod
   - `kubectl logs -f <nom-pod>` - Suivre les journaux
9. **kubectl create namespace <nom>** - Crée un nouveau namespace
10. **kubectl delete pod <nom-pod>** - Supprime un pod

Commandes Kubernetes Intermédiaires

11. **kubectl apply -f <fichier.yaml>** - Applique les changements depuis un fichier YAML
12. **kubectl delete -f <fichier.yaml>** - Supprime les ressources depuis un fichier YAML
13. **kubectl scale deployment <nom> --replicas=3** - Met à l'échelle un déploiement
14. **kubectl expose deployment <nom> --type=LoadBalancer --port=80** - Expose un déploiement
15. **kubectl exec -it <nom-pod> -- /bin/bash** - Exécute une commande dans un pod
16. **kubectl port-forward <nom-pod> 8080:80** - Redirige un port local vers un pod
17. **kubectl get configmaps** - Liste toutes les ConfigMaps
18. **kubectl get secrets** - Liste tous les Secrets
19. **kubectl edit <ressource>/<nom>** - Édite une définition de ressource
20. **kubectl rollout status deployment/<nom>** - Affiche le statut du déploiement

Commandes Kubernetes Avancées

21. **kubectl rollout undo deployment/<nom>** - Annule un déploiement
22. **kubectl top nodes** - Affiche l'utilisation des ressources pour les nœuds
23. **kubectl top pods** - Affiche l'utilisation des ressources pour les pods
24. **kubectl cordon <nom-nœud>** - Marque un nœud comme non planifiable

25. **kubectl uncordon <nom-nœud>** - Marque un nœud comme planifiable
26. **kubectl drain <nom-nœud> --ignore-daemonsets** - Évacue en toute sécurité tous les pods d'un nœud
27. **kubectl taint nodes <nom-nœud> <clé>=<valeur>:<effet>** - Ajoute une taint à un nœud
28. **kubectl get events** - Liste tous les événements
29. **kubectl apply -k <rep>** - Applique les ressources depuis un répertoire kustomization
30. **kubectl config view** - Affiche le fichier kubeconfig
31. **kubectl config use-context <nom-cluster>** - Bascule le contexte actif
32. **kubectl debug pod/<nom-pod>** - Crée une session de débogage pour un pod
33. **kubectl delete namespace <nom>** - Supprime un namespace
34. **kubectl patch <ressource> <nom> -p '{"spec": ...}'** - Met à jour une ressource
35. **kubectl rollout history deployment/<nom>** - Affiche l'historique des déploiements
36. **kubectl autoscale deployment <nom> --cpu-percent=50 --min=1 --max=10** - Crée un HorizontalPodAutoscaler
37. **kubectl label pod <nom-pod> <clé>=<valeur>** - Ajoute ou modifie une étiquette
38. **kubectl annotate pod <nom-pod> <clé>=<valeur>** - Ajoute ou modifie une annotation
39. **kubectl delete pv <nom-pv>** - Supprime un PersistentVolume
40. **kubectl get ingress** - Liste toutes les ressources Ingress
41. **kubectl create configmap <nom> --from-literal=<clé>=<valeur>** - Crée une ConfigMap
42. **kubectl create secret generic <nom> --from-literal=<clé>=<valeur>** - Crée un Secret
43. **kubectl api-resources** - Liste toutes les ressources API disponibles
44. **kubectl api-versions** - Liste toutes les versions d'API
45. **kubectl get crds** - Liste toutes les Custom Resource Definitions (CRDs)

Exemples du Manuel Kubernetes (Manifestes YAML)

Exemple 1: deployment.yaml

Ce manifeste crée un Deployment qui exécute 3 répliques du conteneur Nginx.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
```

```
    labels:
      app: nginx
  spec:
    containers:
    - name: nginx
      image: nginx:1.23
      ports:
      - containerPort: 80
```

Pour appliquer: `kubectl apply -f deployment.yaml`

Exemple 2: service.yaml

Ce manifeste crée un Service de type NodePort pour exposer le nginx-deployment à l'extérieur du cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx # Ceci DOIT correspondre aux labels dans le template du Deployment
  ports:
  - protocol: TCP
    port: 80 # Port sur lequel le service est disponible *à l'intérieur* du cluster
    targetPort: 80 # Port sur lequel le conteneur écoute
    # nodePort: 30080 # Optionnel: spécifier un port (30000-32767)
```

Pour appliquer: `kubectl apply -f service.yaml`

---

5. Helm (Le Gestionnaire de Paquets K8s)

Helm simplifie l'installation et la gestion d'applications Kubernetes complexes en utilisant des "charts".

Commandes Helm de Base

1. **helm help** - Affiche l'aide

2. **helm version** - Affiche la version client/serveur de Helm
3. **helm repo add <nom> <url>** - Ajoute un nouveau dépôt de charts
4. **helm repo update** - Met à jour tous les dépôts de charts
5. **helm repo list** - Liste tous les dépôts ajoutés
6. **helm search hub <mot-clé>** - Recherche des charts sur Helm Hub
7. **helm search repo <mot-clé>** - Recherche des charts dans vos dépôts
8. **helm show chart <depot/chart>** - Affiche des informations sur un chart

Installation et Mise à Niveau des Charts

9. **helm install <nom-release> <depot/chart>** - Installe un chart
10. **helm upgrade <nom-release> <depot/chart>** - Met à niveau une release existante
11. **helm upgrade --install <nom-release> <depot/chart>** - Installe ou met à niveau
12. **helm uninstall <nom-release>** - Désinstalle une release
13. **helm list** - Liste toutes les releases installées
    - `helm list -n <namespace>` ou `helm list -A`
14. **helm status <nom-release>** - Affiche le statut d'une release

Travail avec les Charts Helm

15. **helm create <nom-chart>** - Crée un nouveau répertoire de chart Helm
16. **helm lint ./<nom-chart>** - Vérifie un chart pour les erreurs
17. **helm package ./<nom-chart>** - Emballe un chart dans un fichier .tgz
18. **helm template <nom-release> ./<nom-chart>** - Génère les fichiers YAML sans installer
19. **helm dependency update ./<nom-chart>** - Met à jour les dépendances

Commandes Helm Avancées

20. **helm rollback <nom-release> <numéro_révision>** - Annule une release
21. **helm history <nom-release>** - Affiche l'historique d'une release
22. **helm get all <nom-release>** - Obtient toutes les informations d'une release
23. **helm get values <nom-release>** - Affiche les valeurs utilisées dans une release
24. **helm test <nom-release>** - Exécute les tests définis dans un chart

Dépôts de Charts Helm

25. **helm repo remove <nom>** - Supprime un dépôt de charts
26. **helm repo index <rep>** - Crée un fichier d'index pour un dépôt de charts

Valeurs et Personnalisation Helm

27. **helm install <nom> <chart> --values <values.yaml>** - Installe avec des valeurs personnalisées

28. **helm upgrade <nom> <chart> -f <values.yaml>** - Met à niveau avec des valeurs personnalisées
29. **helm install <nom> <chart> --set <clé>=<valeur>** - Installe avec une valeur directe
30. **helm upgrade <nom> <chart> --set <clé>=<valeur>** - Met à niveau avec une valeur directe

Template et Débogage Helm

31. **helm template <nom> <chart> --debug** - Génère les templates avec sortie de débogage
32. **helm install <nom> <chart> --dry-run** - Simule une installation
33. **helm upgrade <nom> <chart> --dry-run** - Simule une mise à niveau

Intégration Helm et Kubernetes

34. **helm list --namespace <ns>** - Liste les releases dans un namespace
35. **helm uninstall <nom> --namespace <ns>** - Désinstalle depuis un namespace
36. **helm install <nom> <chart> --namespace <ns>** - Installe dans un namespace
37. **helm upgrade <nom> <chart> --namespace <ns>** - Met à niveau dans un namespace

Développement de Charts Helm

38. **helm package --sign** - Emballe et signe un chart
39. **helm create --starter <chemin>** - Crée un chart depuis un template de démarrage
40. **helm push <chart.tgz> <nom_depot>** - Pousse un chart vers un dépôt

Helm avec CLI Kubernetes

41. **helm install <nom> <chart> --kube-context <contexte>** - Installe vers un contexte de cluster spécifique
42. **helm upgrade <nom> <chart> --kube-context <contexte>** - Met à niveau dans un contexte spécifique

Dépendances des Charts Helm

43. **helm dependency build ./<nom-chart>** - Construit les dépendances
44. **helm dependency list ./<nom-chart>** - Liste toutes les dépendances

Historique et Retours Arrière Helm

45. **helm rollback <nom> <révision> --recreate-pods** - Annule et recrée les pods
46. **helm history <nom> --max <nombre>** - Limite la sortie de l'historique

Workflow du Manuel Helm

Workflow: Installer une Stack Prometheus Personnalisée

1. Ajouter le dépôt de la communauté Prometheus:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

2. Rechercher le chart kube-prometheus-stack:

```
helm search repo prometheus-community/kube-prometheus-stack
```

3. Obtenir les valeurs par défaut et les enregistrer dans un fichier:

```
helm show values prometheus-community/kube-prometheus-stack > prom-values.yaml
```

4. Éditer le fichier prom-values.yaml:
    - Par exemple, vous pourriez vouloir désactiver Grafana ou définir la persistance
    - `nano prom-values.yaml`
    - Trouver `grafana:` et définir `enabled: false`
5. Installer le chart dans un namespace monitoring:

```
kubectl create namespace monitoring
helm install my-prometheus prometheus-community/kube-prometheus-stack \
  -n monitoring \
  -f prom-values.yaml
```

6. Vérifier le statut:

```
helm status my-prometheus -n monitoring
kubectl get pods -n monitoring
```

7. Désinstaller la release:

```
helm uninstall my-prometheus -n monitoring
```

---

6. Terraform (Infrastructure as Code)

Terraform vous permet de construire, modifier et versionner l'infrastructure cloud et on-premise de manière sûre et efficace.

Commandes Terraform de Base

1. **terraform --help** - Affiche l'aide générale
2. **terraform init** - Initialise le répertoire de travail (télécharge les providers)
3. **terraform validate** - Valide la syntaxe des fichiers de configuration
4. **terraform plan** - Crée un plan d'exécution
5. **terraform apply** - Applique les changements
   - `terraform apply -auto-approve` - Appliquer sans approbation interactive
6. **terraform show** - Affiche l'état actuel
7. **terraform output** - Affiche les valeurs de sortie
8. **terraform destroy** - Détruit l'infrastructure
9. **terraform refresh** - Met à jour le fichier d'état avec l'infrastructure réelle
10. **terraform taint <adresse_ressource>** - Marque une ressource pour recréation
11. **terraform untaint <adresse_ressource>** - Supprime la marque d'une ressource taintée
12. **terraform state** - Gère les fichiers d'état
13. **terraform import <adresse_ressource> <id_ressource>** - Importe l'infrastructure existante
14. **terraform graph** - Génère une représentation graphique
15. **terraform providers** - Liste les providers
16. **terraform state list** - Liste toutes les ressources dans l'état
17. **terraform backend** - Configure le backend d'état
18. **terraform state mv <source> <destination>** - Déplace un élément dans l'état
19. **terraform state rm <adresse_ressource>** - Supprime un élément de l'état
20. **terraform workspace** - Gère les workspaces
21. **terraform workspace new <nom>** - Crée un nouveau workspace
22. **terraform module** - Gère les modules
23. **TF_LOG=DEBUG** - Définir le niveau de log via variable d'environnement
24. **TF_LOG_PATH=<chemin>** - Définir le chemin du fichier de log
25. **terraform login** - Se connecte à Terraform Cloud/Enterprise

Exemple du Manuel Terraform (HCL)

Cet exemple définit un bucket AWS S3.

Fichier: main.tf

```
# 1. Configurer le Provider AWS
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
    version = "~> 5.0"
  }
 }
}

provider "aws" {
 region = "us-east-1"
}

# 2. Définir une variable pour le nom du bucket
variable "bucket_name" {
 description = "Le nom du bucket S3"
 type        = string
 default     = "mon-bucket-tf-unique-12345"
}

# 3. Créer la ressource bucket S3
resource "aws_s3_bucket" "my_bucket" {
 bucket = var.bucket_name

 tags = {
  Name        = "Mon Bucket Terraform"
  Environment = "Dev"
 }
}

# 4. Sortir le nom du bucket
output "bucket_name" {
 value = aws_s3_bucket.my_bucket.bucket
}
```

Workflow Terraform:

1. `terraform init` - Initialise et télécharge le provider AWS
2. `terraform plan` - Montre qu'il va créer 1 bucket S3
3. `terraform apply` - Demande l'approbation, puis crée le bucket
4. `terraform output` - Affiche le nom du bucket créé
5. `terraform destroy` - Demande l'approbation, puis supprime le bucket

---

7. Ansible (Gestion de Configuration)

Ansible est un outil open-source qui automatise le provisionnement logiciel, la gestion de configuration et le déploiement d'applications. Il est sans agent, ce qui signifie qu'il se connecte aux serveurs via SSH.

Concepts Fondamentaux

- **Inventory (Inventaire)**: Un fichier (comme hosts.ini) qui liste les serveurs gérés par Ansible
- **Playbook**: Un fichier YAML qui définit un ensemble de tâches à exécuter sur un serveur
- **Task (Tâche)**: Une action unique à effectuer (ex: installer un paquet, copier un fichier)
- **Module**: Le code qu'Ansible exécute pour une tâche (ex: apt, copy, systemd)
- **Role (Rôle)**: Une collection de playbooks, templates et variables pour organiser des configurations complexes

Commandes Courantes

- `ansible --version` - Vérifier la version
- `ansible all -m ping -i inventory.ini` - Pinger tous les hôtes dans l'inventaire (commande ad-hoc)
- `ansible-playbook -i inventory.ini playbook.yml` - Exécuter un playbook
- `ansible-playbook -i inventory.ini playbook.yml --check` - Dry-run: voir ce qui changerait
- `ansible-galaxy install <nom_role>` - Installer un rôle depuis Ansible Galaxy

Exemple de Playbook Ansible (YAML)

Ce playbook installe et démarre Nginx sur un groupe de serveurs web.

Fichier: inventory.ini

```
[webservers]
web1.exemple.com
web2.exemple.com
```

Fichier: playbook.yml

```
---
- name: Configurer les Serveurs Web
  hosts: webservers # Ceci correspond au groupe dans l'inventaire
  become: yes # Cela signifie "exécuter en tant que sudo"

  tasks:
    - name: Installer nginx (Debian/Ubuntu)
```

```
    ansible.builtin.apt:
      name: nginx
      state: present
      update_cache: yes
    when: ansible_os_family == "Debian"

  - name: Installer nginx (RHEL/CentOS)
    ansible.builtin.yum:
      name: nginx
      state: present
    when: ansible_os_family == "RedHat"

  - name: Démarrer et activer le service nginx
    ansible.builtin.systemd:
      name: nginx
      state: started
      enabled: yes

  - name: Copier la page index.html personnalisée
    ansible.builtin.template:
      src: index.html.j2 # Un fichier template
      dest: /var/www/html/index.html
      mode: '0644'
```

Fichier: index.html.j2 (Ceci est un template Jinja2)

```
<html>
  <head><title>Bienvenue</title></head>
  <body>
    <h1>Ce serveur est {{ ansible_hostname }}</h1>
    <p>Géré par Ansible.</p>
  </body>
</html>
```

Pour exécuter: `ansible-playbook -i inventory.ini playbook.yml`

---

8. CI/CD (Intégration/Déploiement Continu)

CI/CD est une pratique qui automatise le pipeline de construction, test et déploiement logiciel. GitHub Actions est un outil populaire intégré directement dans GitHub.

Concepts Fondamentaux (GitHub Actions)

- **Workflow**: Un processus automatisé défini dans un fichier YAML dans le répertoire `.github/workflows/`
- **Event (Événement)**: Le déclencheur d'un workflow (ex: `on: push`, `on: pull_request`)
- **Job**: Un ensemble d'étapes qui s'exécutent sur un runner
- **Step (Étape)**: Une tâche unique (soit une commande shell soit une action pré-construite)
- **Action**: Un morceau de code réutilisable (ex: `actions/checkout@v3`)
- **Runner**: Le serveur (Linux, Windows, macOS) qui exécute le job

Exemple du Manuel CI/CD (GitHub Actions)

Ce workflow se déclenche lors d'un push vers la branche main. Il construit une application Node.js, exécute les tests, construit une image Docker et la pousse vers Docker Hub.

Fichier: .github/workflows/main.yml

```yaml
name: Pipeline CI/CD

# 1. Déclencher le workflow lors d'un push vers la branche 'main'
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  # 2. Job pour construire et tester l'application
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Récupérer le code du dépôt
        uses: actions/checkout@v4

      - name: Configurer Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'

      - name: Installer les dépendances
        run: npm install
```

```yaml
      - name: Exécuter les tests
        run: npm test

  # 3. Job pour construire et pousser l'image Docker
  build-and-push-docker:
    needs: build-and-test # Ce job ne s'exécute que si 'build-and-test' réussit
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' # S'exécute uniquement sur push vers main, pas sur les PRs

    steps:
      - name: Récupérer le code du dépôt
        uses: actions/checkout@v4

      - name: Se connecter à Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Construire et pousser l'image Docker
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: votre-nom-dockerhub/mon-app:latest
```

Pour utiliser:

1. Ajouter ce fichier à votre dépôt à `.github/workflows/main.yml`
2. Ajouter `DOCKER_USERNAME` et `DOCKER_PASSWORD` aux Secrets de votre dépôt GitHub dans Settings > Secrets and variables > Actions secrets

---

9. Surveillance et Observabilité

- **Surveillance (Monitoring)**: Vous indique si quelque chose ne va pas (ex: le CPU est à 90%)
- **Observabilité (Observability)**: Vous indique pourquoi quelque chose ne va pas (ex: une fonction spécifique est en boucle)

Outils Principaux

- **Prometheus**: Une base de données de séries temporelles qui récupère (scrape) les métriques de vos applications
- **Grafana**: Un outil de visualisation qui interroge Prometheus (et d'autres sources) pour créer des tableaux de bord
- **Stack ELK/EFK**:
  - **Elasticsearch**: Une base de données pour stocker les logs
  - **Logstash / Fluentd**: Outils pour collecter et traiter les logs
  - **Kibana**: Un outil de visualisation pour les logs

Prometheus & PromQL

Prometheus utilise un langage de requête puissant appelé PromQL.

Requêtes PromQL Courantes:

- Obtenir le taux par seconde des requêtes HTTP sur les 5 dernières minutes:

```
rate(http_requests_total[5m])
```

- Obtenir le 95e percentile de latence des requêtes:

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

- Montrer combien d'instances de chaque job sont "up" (en cours d'exécution):

```
sum(up) by (job)
```

- Obtenir la mémoire libre sur les nœuds (en utilisant Node Exporter):

```
node_memory_MemFree_bytes
```

Grafana

Grafana est la couche de visualisation. Vous n'exécutez pas de "commandes" dedans, mais vous l'utilisez pour:

1. Ajouter Prometheus comme source de données
2. Créer des tableaux de bord avec des panneaux
3. Écrire des requêtes PromQL dans les panneaux pour construire des graphiques
4. Configurer des alertes basées sur des seuils de requêtes

Journalisation (ELK/EFK)

L'objectif est de centraliser les logs.

1. **Fluentd/Logstash** s'exécute sur vos nœuds (ou comme sidecar dans K8s)
2. Il collecte les logs depuis les fichiers (`/var/log/*.log`) ou la sortie des conteneurs
3. Il analyse, enrichit et transmet ces logs à Elasticsearch
4. **Kibana** fournit une interface web pour rechercher et visualiser tous vos logs en un seul endroit

---

Merci. Connectons-nous

www.linkedin.com/in/bhooshan-pattanashetti